IoT Venture: Wireless Demo & Secure Firmware Updates

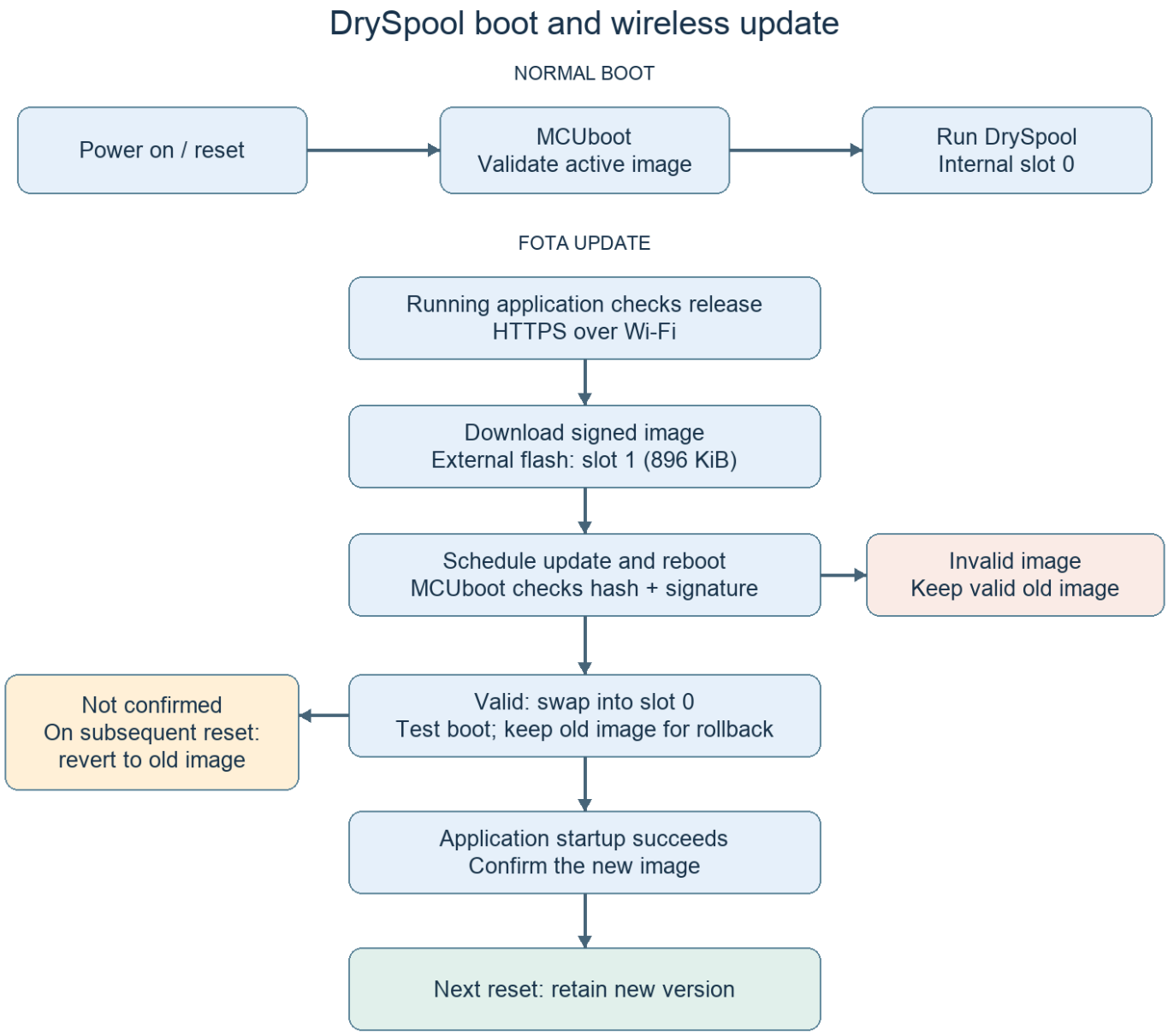
Team: T06 — DrySpool

Members: Sirui Wu, Zicong Zhang, Deepa Lokesha

Platform: nRF7002 DK, nRF Connect SDK 3.4.0

Date: November 2, 2026

Q1. Draw and describe your bootloading process



Initial MCUboot + application installation uses the USB debug probe.

We use MCUboot to validate and boot our application. For an update, the application downloads the new image to external flash and reboots. MCUboot checks the image and swaps it into the primary slot for a test boot. The application then confirms the image. If it is not confirmed, MCUboot restores the previous image on the next reset.

1. How large is your bootloader?

Our documented baseline MCUboot build uses **39,308 bytes (38.4 KiB)**, within a 64 KiB partition.

2. How large is your existing application code?

Our local `0.1.1` build uses **728,024 bytes (711 KiB)** of flash for the non-secure application. The baseline TF-M secure application uses another **130,008 bytes (127 KiB)**. The signed image downloaded in our FOTA test was **858,974 bytes**; it came from a separate cloud release build.

3. Does the bootloader or application code handle the firmware image download?

The **application** downloads the image using the Memfault/NCS FOTA integration. MCUboot validates and installs it after reboot.

4. What wireless communication is used, and why?

We use **HTTPS over Wi-Fi**. Our board also supports BLE, but Wi-Fi lets it download directly from the cloud using the same connection as our product, without a phone acting as a gateway.

5. Where are the downloaded firmware images stored?

The new image is stored in the **896 KiB secondary slot in external MX25R64 QSPI flash**. The active application runs from the primary slot in internal flash.

6. What features handle firmware update failures?

- **Invalid signatures:** MCUboot verifies the ECDSA-P256 signature using our public key and rejects images signed with an untrusted key.
- **Corrupted or incomplete downloads:** Hash validation detects corruption, and incomplete downloads are not scheduled for installation.
- **Faulty code with a valid signature:** We use test boot and image confirmation. If the application does not confirm the image, MCUboot rolls back on the next reset. A hung application still needs a reset.

Our application confirms the image after sensor initialization succeeds. This is a basic startup check using simulated sensors. It does not catch every bug, and automatic rollback no longer applies after confirmation.

Q2. Implement FOTA updates on your existing codebase from the Wireless Demo section

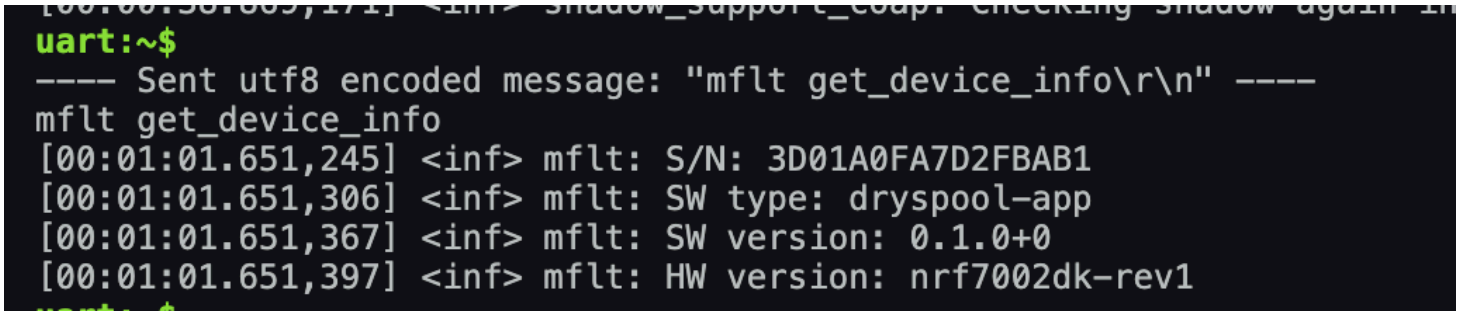
Code: github.com/ese5180/f26-t06-dry-spool

We added FOTA to our existing DrySpool application:

Requirement	Our implementation
MCUboot support in Zephyr	Enabled through `firmware/Kconfig.sysbuild`.
External flash storage	Application and MCUboot overlays configure the secondary image slot.
Signed firmware with custom keys	`firmware/sysbuild.conf` selects our generated ECDSA-P256 signing key. The private key is excluded from Git.
Wireless FOTA	`firmware/coap-fota.conf` and `firmware/src/memfault_fota_support.c` support Wi-Fi downloads and image confirmation.

FOTA test: 0.1.0 → 0.1.1

We tested on device **3D01A0FA7D2FBAB1**. Before the update, `mflt get_device_info` showed **0.1.0+0**.



We checked for a release and started the download from the UART shell:

```
mflt get_latest_url
mflt get_latest_release
```

The firmware was downloaded over Wi-Fi. The log showed a complete download, MCUboot swap, and image confirmation:

```
Downloaded 858974/858974 bytes (100%)
MCUBoot image-0 upgrade scheduled. Reset device to apply
FOTA update downloaded, rebooting to apply it
Image index: 0, Swap type: test
Starting swap using move algorithm.
Image version: v0.1.1
Confirming FOTA update image
```

After the automatic reboot, the same device reported **0.1.1+0**.



The device reconnected to nRF Cloud and successfully uploaded diagnostics. The cloud page also showed **0.1.1+0**:



We rebooted the board again and confirmed that it kept the new version. This completed the wireless update from **0.1.0 to 0.1.1**. We did not separately test wrong-signature rejection or faulty-image rollback.

Q3. Wireless Demo (in person)

Q4. Submission Requirements